

Ejercicio 2:

La Falacia de los Recursos y el Cono de Incertidumbre Contexto: Faltan 2 meses para la fecha límite de un proyecto de software crítico. El proyecto está retrasado. El Director de la Empresa, ignorando la Ley de Brooks, ordena duplicar el equipo de desarrollo (de 5 a 10 programadores) para recuperar el tiempo perdido y cumplir con el cronograma de Gantt original.

Analice matemáticamente y conceptualmente por qué la ecuación $\text{Tiempo} = \text{Esfuerzo} / \text{Recursos}$ falla en este escenario específico.

- a) Detalle, explique tres (3) factores de sobrecarga (comunicación, onboarding, integración) que ocurrirán al duplicar el equipo.
- b) Proponga, explique dos (2) alternativas de gestión (una desde un enfoque predictivo tradicional y otra desde un enfoque ágil/híbrido) para manejar esta crisis de cronograma sin violar los principios de la ingeniería de software.

Respuesta:

La idea se basa en la ecuación:

$$\textit{Tiempo} = \frac{\textit{Esfuerzo}}{\textit{Recursos}}$$

Esta fórmula supone que, al aumentar los recursos, el tiempo disminuye proporcionalmente. Sin embargo, en software esta relación no siempre se cumple, porque el trabajo no puede dividirse de manera perfecta entre todos los programadores. Existen dependencias, decisiones de arquitectura, pruebas, comunicación e integración de código.

El problema principal es que el número de canales de comunicación aumenta mucho más rápido que el número de programadores. La cantidad de posibles canales de comunicación de un equipo se calcula así:

$$C = \frac{n(n-1)}{2} \quad \text{Donde } n \text{ representa el número de integrantes.}$$

Para el equipo original de cinco programadores:

$$C1 = \frac{5(5-1)}{2} = 10$$

Al duplicar el equipo a diez programadores:

$$C2 = \frac{10(10 - 1)}{2} = 45$$

El número de personas aumenta 100%, pero los canales de comunicación aumentan de 10 a 45, lo que representa un incremento de 350%:

$$\frac{45 - 10}{10} \times 100 = 350\%$$

Esto significa que se necesitarán más reuniones, explicaciones, revisiones, coordinación de tareas y resolución de conflictos. Por tanto, la productividad no crece de manera lineal.

Una expresión más cercana a la realidad sería:

$$E_{neto}(n) = n \cdot p - c \cdot \frac{n(n - 1)}{2}$$

En esta fórmula, $n \cdot p$ representa la capacidad de trabajo del equipo, mientras que el segundo término representa el tiempo perdido por comunicación, coordinación y supervisión. Si estos costos son muy altos, agregar programadores puede reducir la productividad efectiva durante las primeras semanas.

Tres (3) Factores de Sobrecarga al Duplicar el Equipo

Onboarding o incorporación

Los cinco programadores nuevos no conocen de inmediato el proyecto. Deben aprender la arquitectura, los requisitos, las herramientas, el repositorio, las normas de código y el funcionamiento de los módulos existentes.

El problema es que los desarrolladores originales deben detener parte de sus tareas para capacitar, responder preguntas y revisar el trabajo de los nuevos integrantes. Como solo faltan ocho semanas, el tiempo de aprendizaje puede consumir una parte importante del plazo disponible.

Comunicación y coordinación

Al aumentar el equipo de cinco a diez personas, los canales de comunicación pasan de 10 a 45. Esto obliga a realizar más reuniones, intercambiar más mensajes y aclarar más dudas sobre requisitos, prioridades y decisiones técnicas.

Además, puede ocurrir que dos personas desarrollen tareas similares o que un programador modifique una parte del sistema sin informar a quienes trabajan en módulos relacionados. Esto genera confusión, errores y retrasos.

Integración y pruebas

Con más desarrolladores trabajando al mismo tiempo, aumentan los conflictos al integrar el código. Pueden aparecer cambios simultáneos en los mismos archivos, errores de compatibilidad, diferencias en el estilo de programación y fallas en pruebas de integración.

También se incrementa el trabajo de revisión de código, corrección de errores y pruebas de regresión. Un cambio realizado para resolver una tarea puede afectar otros módulos que ya funcionaban correctamente. Por esta razón, agregar desarrolladores sin planificación puede causar más retrabajo que avance real.

Dos (2) Alternativas de Gestión sin Violar Principios de la Ingeniería de Software

Enfoque predictivo tradicional

Desde un enfoque predictivo, la mejor opción es revisar formalmente el cronograma, identificar la ruta crítica y determinar cuáles tareas afectan directamente la fecha final. No todas las actividades tienen la misma importancia; asignar recursos a tareas con holgura no reduce la duración total del proyecto.

Si el plazo de dos meses no puede modificarse, la organización debe negociar una reducción controlada del alcance. Esto significa conservar las funcionalidades esenciales y trasladar las menos importantes a una fase posterior.

Enfoque ágil o híbrido

Desde un enfoque ágil o híbrido, la alternativa es mantener fijo el plazo de entrega y ajustar el alcance del producto. Para ello, se debe revisar el *Product Backlog* junto con el cliente y clasificar las funcionalidades usando la técnica MoSCoW:

- **Must Have:** funciones indispensables para que el sistema opere.
- **Should Have:** funciones importantes, pero que pueden esperar.
- **Could Have:** mejoras deseables, pero no necesarias.
- **Won't Have:** funciones que se eliminan de esta entrega.

Con esta priorización se construye un **Producto Mínimo Viable** o MVP. El objetivo no es entregar un producto incompleto, sino una primera versión funcional, estable y probada con las características esenciales.